

# Guía básica de GitHub

Para ayudantes de Economía Política

Cátedra de Economía Política, UNC

2026-01-01

## Tabla de contenidos

|          |                                                                               |          |
|----------|-------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Presentación</b>                                                           | <b>1</b> |
| 1.1      | 1. Qué es cada cosa . . . . .                                                 | 2        |
| 1.2      | 2. Instalación inicial . . . . .                                              | 2        |
| 1.3      | 3. Mapa visual del flujo de trabajo . . . . .                                 | 3        |
| 1.4      | 4. Clonar el repositorio del curso con GitHub Desktop . . . . .               | 3        |
| 1.5      | 5. Regla de oro antes de trabajar . . . . .                                   | 4        |
| 1.6      | 6. Cómo hacer cambios correctamente . . . . .                                 | 4        |
| 1.7      | 7. Cómo hacer un commit con GitHub Desktop . . . . .                          | 5        |
| 1.8      | 8. Cómo subir cambios con push . . . . .                                      | 6        |
| 1.9      | 9. Cómo bajar cambios de otras personas . . . . .                             | 6        |
| 1.10     | 10. Flujo alternativo por terminal . . . . .                                  | 6        |
| 1.11     | 11. Reglas básicas de convivencia en el repositorio . . . . .                 | 8        |
| 1.12     | 12. Nombres recomendados para archivos y carpetas . . . . .                   | 8        |
| 1.13     | 13. Qué hacer si aparece un conflicto . . . . .                               | 9        |
| 1.14     | 14. Ejemplo completo con GitHub Desktop: corregir una guía práctica . . . . . | 10       |
| 1.15     | 15. Ejemplo completo con terminal: corregir una guía práctica . . . . .       | 10       |
| 1.16     | 16. Ejemplo completo: agregar filminas nuevas . . . . .                       | 10       |
| 1.17     | 17. Capturas y materiales visuales sugeridos . . . . .                        | 11       |
| 1.18     | 18. Cosas que conviene no hacer . . . . .                                     | 11       |
| 1.19     | 19. Checklist rápido antes de terminar . . . . .                              | 12       |
| 1.20     | 20. Pedir ayuda . . . . .                                                     | 12       |

## 1 Presentación

Esta guía es para quienes se suman a la cátedra de **Economía Política** y van a colaborar con materiales del repositorio del curso. No hace falta saber programar ni tener experiencia previa con GitHub.

La idea principal es simple: el repositorio es una carpeta compartida, pero con historial. Cada cambio queda registrado, se puede revisar quién hizo qué y se puede volver atrás si algo sale mal.

Esta guía presenta **dos formas de trabajar**:

1. **GitHub Desktop**, recomendada para empezar porque muestra los cambios de manera visual.

2. **Terminal**, recomendada para quienes quieran un flujo más rápido o ya se sientan cómodos escribiendo comandos.

No hace falta elegir una para siempre. Se puede empezar con GitHub Desktop y más adelante pasar a terminal.

#### Idea clave

El hábito más importante es siempre el mismo: **hacer pull antes de editar y hacer push cuando terminás una tarea concreta.**

## 1.1 1. Qué es cada cosa

- **GitHub**: la página web donde está guardado el repositorio de la cátedra.
- **Repositorio**: la carpeta compartida del curso. En este caso, `econ-pol`.
- **Git**: el sistema que registra los cambios en los archivos.
- **Clonar**: bajar una copia del repositorio a tu computadora.
- **Pull**: traer a tu computadora los últimos cambios que hicieron otras personas.
- **Commit**: guardar un paquete de cambios con un mensaje explicativo.
- **Push**: subir tus commits a GitHub para que el resto los vea.
- **Conflicto**: pasa cuando dos personas editaron la misma parte de un archivo y Git no sabe cuál versión conservar.

## 1.2 2. Instalación inicial

### 1.2.1 Paso 1: crear cuenta de GitHub

1. Entrá a <https://github.com/>.
2. Creá una cuenta con un mail que uses seguido.
3. Avisá tu usuario de GitHub a la cátedra para que te agreguen al repositorio.

### 1.2.2 Paso 2: instalar GitHub Desktop

Para empezar desde cero, recomendamos usar **GitHub Desktop** porque evita casi todo el trabajo con la terminal. Al principio puede resultar un poco raro porque usa palabras nuevas, pero el flujo cotidiano es bastante repetitivo: traer cambios, editar, revisar, guardar commit y subir.

1. Entrá a <https://desktop.github.com/>.
2. Descargá e instalá GitHub Desktop.
3. Abrí GitHub Desktop.
4. Iniciá sesión con tu cuenta de GitHub.

### 1.2.3 Paso 3: instalar Git

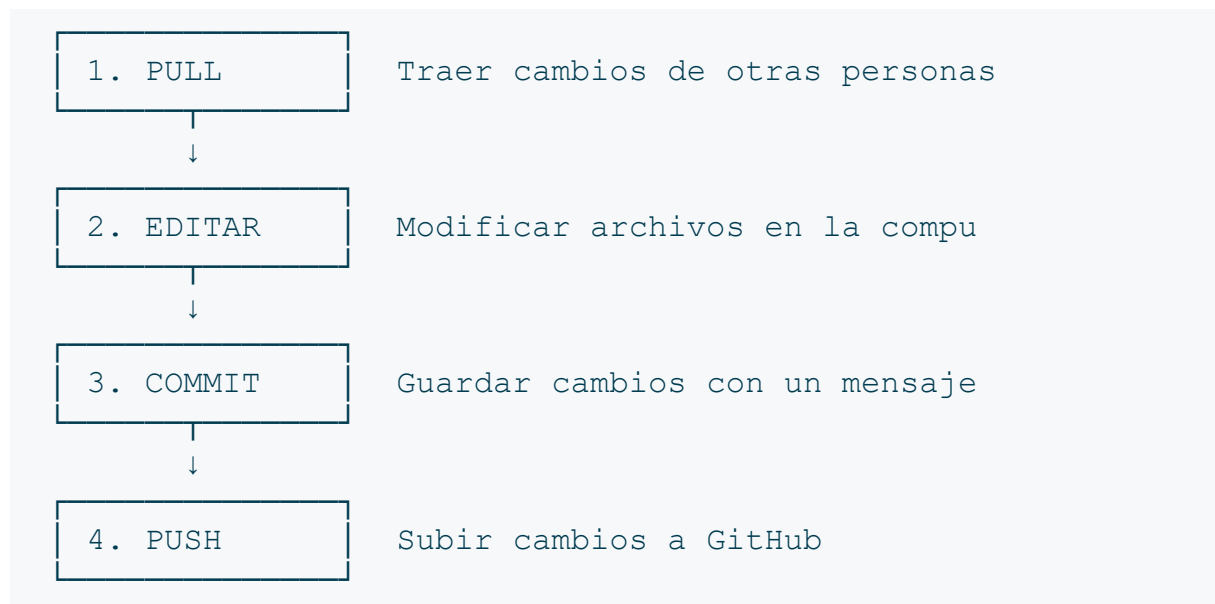
GitHub Desktop suele instalar o detectar Git automáticamente. Si te pide instalar Git, seguí las instrucciones del programa.

Si querés instalarlo manualmente:

1. Entrá a <https://git-scm.com/downloads>.
2. Descargá la versión para tu sistema operativo.
3. Durante la instalación, dejá las opciones por defecto.

### 1.3 3. Mapa visual del flujo de trabajo

La rutina completa, tanto con GitHub Desktop como con terminal, es esta:



Versión ultra corta:

```
Pull -> editar -> commit -> push
```

Si dudás, volvé a esta secuencia.

### 1.4 4. Clonar el repositorio del curso con GitHub Desktop

Clonar significa bajar una copia local del repositorio a tu computadora.

1. Abrí GitHub Desktop.
2. Andá a File > Clone repository.
3. Elegí el repositorio econ-pol.
4. Elegí una carpeta clara en tu computadora, por ejemplo:
  - Documentos/GitHub/econ-pol
  - Desktop/GitHub/econ-pol
5. Hací clic en Clone.

Desde ese momento vas a tener una carpeta local con los materiales del curso.

### 1.4.1 Mini guía visual: pantalla principal de GitHub Desktop

Cuando abras GitHub Desktop, prestá atención a estas zonas:

|                                  |                               |                       |
|----------------------------------|-------------------------------|-----------------------|
| Repositorio actual: econ-pol     |                               | Botón Fetch/Pull/Push |
| Archivos modificados             | Vista del cambio seleccionado |                       |
| docs/guia.md                     | Líneas agregadas o eliminadas |                       |
| slides/clase.qmd                 |                               |                       |
| Mensaje del commit               |                               |                       |
| [Summary: Corrige guía práctica] |                               |                       |
| [Commit to main]                 |                               |                       |

Traducción de botones importantes:

- Fetch origin: pregunta si hay cambios nuevos en GitHub.
- Pull origin: baja esos cambios a tu computadora.
- Commit to main: guarda tus cambios en el historial local.
- Push origin: sube tus commits a GitHub.
- Changes: muestra archivos modificados.
- History: muestra commits anteriores.

## 1.5 5. Regla de oro antes de trabajar

Antes de modificar cualquier archivo, hacé siempre:

```
Pull
```

En GitHub Desktop aparece como Fetch origin o Pull origin, según el caso.

Esto sirve para traer los cambios más recientes y evitar pisar el trabajo de otra persona.

La rutina ideal en GitHub Desktop es:

1. Abrir GitHub Desktop.
2. Hacer Fetch origin.
3. Si aparece la opción, hacer Pull origin.
4. Recién ahí abrir los archivos y trabajar.

En terminal, la misma rutina es:

```
git pull
```

## 1.6 6. Cómo hacer cambios correctamente

Cuando quieras editar materiales:

1. Abrió la carpeta del repositorio en tu computadora.
2. Buscá el archivo que tenés que modificar.
3. Editalo con el programa correspondiente:
  - .qmd: Positron, RStudio, VS Code o editor de texto.
  - .R: RStudio, Positron o VS Code.
  - .docx: Word o LibreOffice.
  - .xlsx: Excel o LibreOffice.
  - .pdf: normalmente no se edita directamente; se reemplaza por una nueva versión si corresponde.
4. Guardá el archivo.
5. Volvé a GitHub Desktop.
6. Revisá la lista de cambios.

No hace falta subir cada archivo inmediatamente, pero sí conviene hacer commits frecuentes cuando terminás una tarea concreta.

## 1.7 7. Cómo hacer un commit con GitHub Desktop

Un commit es como decir: “terminé este cambio y lo guardo en el historial”.

En GitHub Desktop:

1. Mirá la lista de archivos modificados.
2. Tildá solo los archivos que querés incluir.
3. Escribí un mensaje corto y claro en Summary.
4. Si hace falta, agregá más detalle en Description.
5. Hacé clic en Commit to main.

Ejemplos de buenos mensajes:

- Corrige errores de tipeo en clase de democracia
- Agrega guía práctica sobre votación
- Actualiza gráficos del práctico 2024
- Sube consignas del trabajo final

Ejemplos de malos mensajes:

- cambios
- cosas
- actualización
- final final ahora sí

### 1.7.1 Ejemplo visual de commit en GitHub Desktop

Supongamos que corregiste una guía. En GitHub Desktop deberías ver algo parecido a esto:

```
Changes
□ Guia practica/guia-instituciones.docx

Summary
```

```
Corrige guía práctica de instituciones
```

```
Description
```

```
Corrige consignas y errores de tipeo detectados en clase.
```

```
[Commit to main]
```

Antes de tocar `Commit to main`, revisá dos cosas:

- que estén tildados solo los archivos que querés incluir;
- que el mensaje explique qué hiciste.

### 1.8 8. Cómo subir cambios con push

Después de hacer commit, los cambios todavía están solo en tu computadora.

Para subirlos a GitHub:

1. En GitHub Desktop, hacé clic en `Push origin`.
2. Esperá a que termine.
3. Si no aparece error, listo: el resto ya puede ver tus cambios.

La secuencia completa en GitHub Desktop es:

```
Pull -> editar -> guardar -> commit -> push
```

### 1.9 9. Cómo bajar cambios de otras personas

Si otra persona subió cambios, vos los bajás con:

```
Fetch origin -> Pull origin
```

Hacelo siempre:

- antes de empezar a trabajar;
- después de varios días sin abrir el repo;
- antes de subir cambios propios;
- cuando alguien avise que subió materiales nuevos.

### 1.10 10. Flujo alternativo por terminal

La terminal no es obligatoria, pero es útil. Si usás terminal, abrila dentro de la carpeta del repositorio `econ-pol`.

### 1.10.1 Primera vez: clonar el repositorio

```
git clone https://github.com/rfhfmnn/econ-pol.git
cd econ-pol
```

Esto crea una carpeta llamada `econ-pol` y entra en ella.

### 1.10.2 Antes de empezar a trabajar

```
git pull
```

Esto baja los últimos cambios.

### 1.10.3 Ver qué archivos cambiaste

```
git status
```

Si ves archivos en rojo o verde, Git detectó cambios.

### 1.10.4 Agregar archivos al próximo commit

Para agregar un archivo puntual:

```
git add ruta/del/archivo.qmd
```

Para agregar todos los cambios de la carpeta actual:

```
git add .
```

Usá `git add .` con cuidado: puede incluir archivos que no querías subir.

### 1.10.5 Crear el commit

```
git commit -m "Corrige guía práctica de instituciones"
```

El mensaje va entre comillas y tiene que explicar el cambio.

### 1.10.6 Subir cambios a GitHub

```
git push
```

### 1.10.7 Flujo completo por terminal

```
git pull
git status
git add ruta/del/archivo.qmd
git commit -m "Describe el cambio realizado"
git push
```

### 1.10.8 Cuándo usar terminal y cuándo GitHub Desktop

Usá **GitHub Desktop** si:

- estás empezando desde cero;
- querés ver los cambios de manera visual;
- te resulta más cómodo hacer clics que escribir comandos.

Usá **terminal** si:

- ya entendés el flujo básico;
- querés trabajar más rápido;
- necesitás copiar y pegar instrucciones precisas;
- alguien te está ayudando y te pasa comandos concretos.

## 1.11 11. Reglas básicas de convivencia en el repositorio

Para evitar líos, sigamos estas reglas:

- Hacer Pull antes de editar.
- No borrar carpetas ni archivos sin consultar.
- No mover muchos archivos de lugar sin avisar.
- No subir datos sensibles de estudiantes.
- No subir contraseñas, tokens ni claves privadas.
- No editar el mismo archivo al mismo tiempo que otra persona si se puede evitar.
- Hacer commits chicos y claros, no un mega-commit con veinte cosas mezcladas.
- Avisar por el canal de la cátedra cuando se modifica un material importante.

## 1.12 12. Nombres recomendados para archivos y carpetas

Para nuevos archivos, conviene usar nombres simples:

- sin espacios;
- sin acentos;
- todo en minúscula;
- con guiones medios.

Ejemplos recomendados:

```
clase-democracia.qmd  
practico-votacion-2026.qmd  
guia-trabajo-final.docx  
imagenes-democracia/
```

Ejemplos a evitar:

```
Clase Democracia FINAL ahora sí.qmd  
práctico votación versión nueva.docx  
imagenes nuevas varias/
```

Esto ayuda a que los archivos funcionen bien en distintas computadoras.

### 1.13 13. Qué hacer si aparece un conflicto

Un conflicto no significa que rompiste todo. Significa que Git necesita ayuda para decidir qué versión conservar.

Puede pasar si:

- dos personas editaron el mismo archivo;
- alguien movió un archivo mientras otra persona lo editaba;
- trabajaste varios días sin hacer Pull.

Si aparece un conflicto y no sabés qué hacer:

1. No entres en pánico.
2. No borres archivos al azar.
3. Sacá captura del mensaje si hace falta.
4. Avisá a la cátedra o a alguien con más experiencia.
5. Explicá qué archivo estabas editando y qué cambio querías subir.

Con conflictos, mejor preguntar antes que intentar arreglarlo a ciegas.

#### 1.13.1 Cómo se ve un conflicto en GitHub Desktop

GitHub Desktop puede mostrar un mensaje parecido a:

```
This branch has conflicts that must be resolved
```

O puede marcar archivos como conflicted.

En ese caso:

- no hagas commits nuevos;
- no borres archivos para “limpiar”;
- no aceptes cambios sin leer;
- pedí ayuda y avisá qué estabas intentando hacer.

### 1.13.2 Cómo se ve un conflicto en terminal

En terminal puede aparecer algo parecido a:

```
CONFLICT (content): Merge conflict in archivo.qmd
Automatic merge failed; fix conflicts and then commit the result.
```

Si aparece eso y no tenés experiencia, frená y pedí ayuda.

## 1.14 14. Ejemplo completo con GitHub Desktop: corregir una guía práctica

Supongamos que tenés que corregir errores en una guía.

1. Abrís GitHub Desktop.
2. Hacés `Fetch origin`.
3. Si aparece, hacés `Pull origin`.
4. Abrís la guía en Word, LibreOffice o el editor correspondiente.
5. Corregís el texto.
6. Guardás el archivo.
7. Volvés a GitHub Desktop.
8. Revisás que aparezca solo ese archivo modificado.
9. Escribís el commit:

```
Corrige guía práctica de instituciones
```

10. Hacés `Commit to main`.
11. Hacés `Push origin`.
12. Avisás: "Subí correcciones de la guía práctica de instituciones".

## 1.15 15. Ejemplo completo con terminal: corregir una guía práctica

Supongamos que editaste este archivo:

```
Guia practica/guia-instituciones.docx
```

El flujo sería:

```
git pull
git status
git add "Guia practica/guia-instituciones.docx"
git commit -m "Corrige guía práctica de instituciones"
git push
```

Después avisás al resto que ya subiste los cambios.

## 1.16 16. Ejemplo completo: agregar filminas nuevas

Supongamos que preparaste nuevas filminas en Quarto.

1. Hacés `Pull` antes de empezar.
2. Agregás el archivo `.qmd` en la carpeta correspondiente.
3. Agregás las imágenes necesarias en una subcarpeta clara.
4. Revisás que las rutas a imágenes funcionen.
5. Renderizás o exportás el PDF si la cátedra lo necesita.
6. En GitHub Desktop, revisás los archivos nuevos.
7. Hacés un commit con mensaje claro:

```
Agrega filminas sobre medios y politica
```

8. Hacés `Push origin`.
9. Avisás al resto.

### 1.17 17. Capturas y materiales visuales sugeridos

Para una capacitación presencial o por Zoom, conviene mostrar estas capturas de pantalla:

1. GitHub Desktop recién abierto con el repositorio `econ-pol` seleccionado.
2. Botón `Fetch origin` antes de empezar a trabajar.
3. Lista de archivos modificados en la pestaña `Changes`.
4. Caja de mensaje del commit con un ejemplo bien escrito.
5. Botón `Push origin` después del commit.
6. Ejemplo de `git status` en terminal.

También puede servir grabar un video corto o GIF con esta secuencia:

```
Abrir GitHub Desktop -> Fetch/Pull -> editar archivo -> ver Changes ->
```

Si se agregan capturas reales más adelante, guardarlas en una carpeta como:

```
docs/img/github-desktop/
```

con nombres simples, por ejemplo:

```
01-fetch-origin.png  
02-changes.png  
03-commit-message.png  
04-push-origin.png
```

### 1.18 18. Cosas que conviene no hacer

- No trabajar directamente sobre archivos si no hiciste `Pull` antes.
- No subir archivos duplicados con nombres como `final`, `final2`, `nuevo`, `ahora-si`.
- No usar el repositorio como depósito general de archivos personales.
- No subir bases de datos grandes sin consultar.
- No resolver conflictos sin entender qué estás aceptando o descartando.
- No cambiar la estructura general del repositorio sin coordinación previa.

### 1.19 19. Checklist rápido antes de terminar

Antes de cerrar la computadora, revisá:

- ¿Hice commit de los cambios que quería guardar?
- ¿El mensaje del commit se entiende?
- ¿Hice push?
- ¿GitHub Desktop quedó sin cambios pendientes raros?
- ¿Avisé si modifiqué algo importante?

### 1.20 20. Pedir ayuda

Cuando pidas ayuda, tratá de mandar esta información:

- qué estabas intentando hacer;
- qué archivo tocaste;
- qué mensaje apareció;
- si ya habías hecho commit o push;
- captura de pantalla si hay error.

La mayoría de los problemas de GitHub tienen arreglo. Lo importante es no borrar cosas a ciegas y avisar rápido.